Report: Parallel D2Q9 Lattice Boltzmann Solver

Author: Zakaria Bargach, 5990470

LBM Simple Solver · coarray Fortran · BGK collision · D2Q9 lattice

This report evaluates two questions:

1. Does the solver reproduce the expected physical behaviour?
2. How well does the current coarray implementation perform and scale?

The evidence is presented before the numerical theory. All tables and plots below are generated from archived result files in this repository, so the notebook can be rerun when new measurements are added.

Summary

The solver passes its main validation checks and reaches a measured peak of **3.16 GLUPS** in the available cluster data. The most important findings are:

- The shear-wave sweep recovers the imposed kinematic viscosity to within **0.0264% in the worst case**, well inside the 1% validation threshold.
- Couette and Poiseuille flow reproduce the expected linear and parabolic velocity profiles. The converged Poiseuille run has a relative L2 error of 1.11×10^{-4}.
- Four of five 300×300 lid-driven-cavity runs converge to a residual below 10^{-8}. The Re=5000 case reaches its 1.6-million-step limit and must be treated as a bounded, non-converged result.
- The largest recorded median throughput is **3157 MLUPS** for 1200×1200 on 256 coarray images. A 2000×2000 grid reaches **2596 MLUPS** at the same image count.
- The 300×300 problem has already saturated: throughput falls from **790.7 MLUPS at 128 images to 768.3 MLUPS at 256**.
- Several repeated measurements have wide ranges, and some very short parallel runs finish in only a few seconds. Longer timed regions and a placement study are the highest-value next experiments.

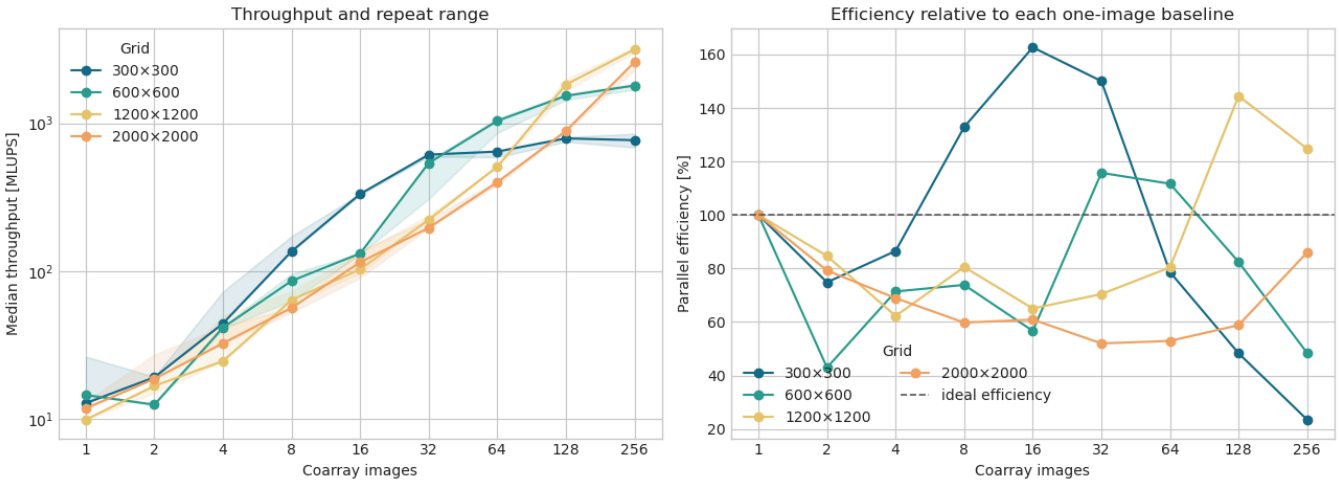
```
(36, 6, 5)
```

1. Cluster performance

The benchmark path runs a fixed 5,000 steps with convergence checks, diagnostics, and field output disabled. The reported wall time is the maximum across images, and throughput is

$$\mathrm{MLUPS} = \frac{N_x N_y N_{\mathrm{steps}}}{10^6 t_{\mathrm{wall}}}.$$

Each plotted point is the median of three runs; the shaded band spans the measured minimum and maximum.




```
Peak: 3157.5 MLUPS on 1200×1200 with 256 images.
```

```
256-image comparison
grid       runtime [s]   MLUPS    efficiency
300×300          0.586    768.3      23.5%
600×600          1.000   1800.4      48.4%
1200×1200         2.280   3157.5     124.8%
2000×2000         7.705   2595.6      86.0%
```

Performance interpretation

The data show a clear problem-size effect. A 300×300 grid gives each image too little work at high image counts, so halo synchronization and communication dominate. The larger grids continue to benefit from additional images through 256.

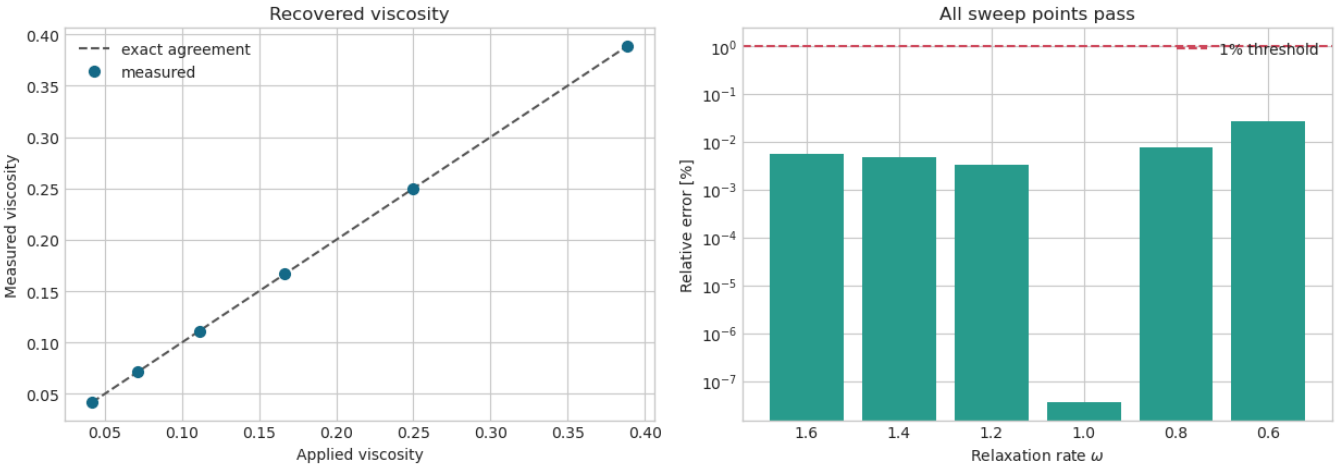
Efficiencies above 100% should not be read as physically superlinear scaling without qualification. The one-image baselines and several intermediate points contain large run-to-run variation, while cache effects can also improve per-core performance after decomposition. In particular, some high-image-count runs last only 1–8 seconds, making startup effects and system noise a significant fraction of the measurement.

```
Five widest repeat ranges, normalized by median MLUPS
grid       images   runtime [s]   relative range
300×300         1      35.194          106.9%
1200×1200       4     292.619          103.3%
300×300         4      10.166           64.3%
600×600        32       3.344           46.9%
2000×2000       2    1070.481           45.6%
```

2. Physical validation

2.1 Shear-wave viscosity

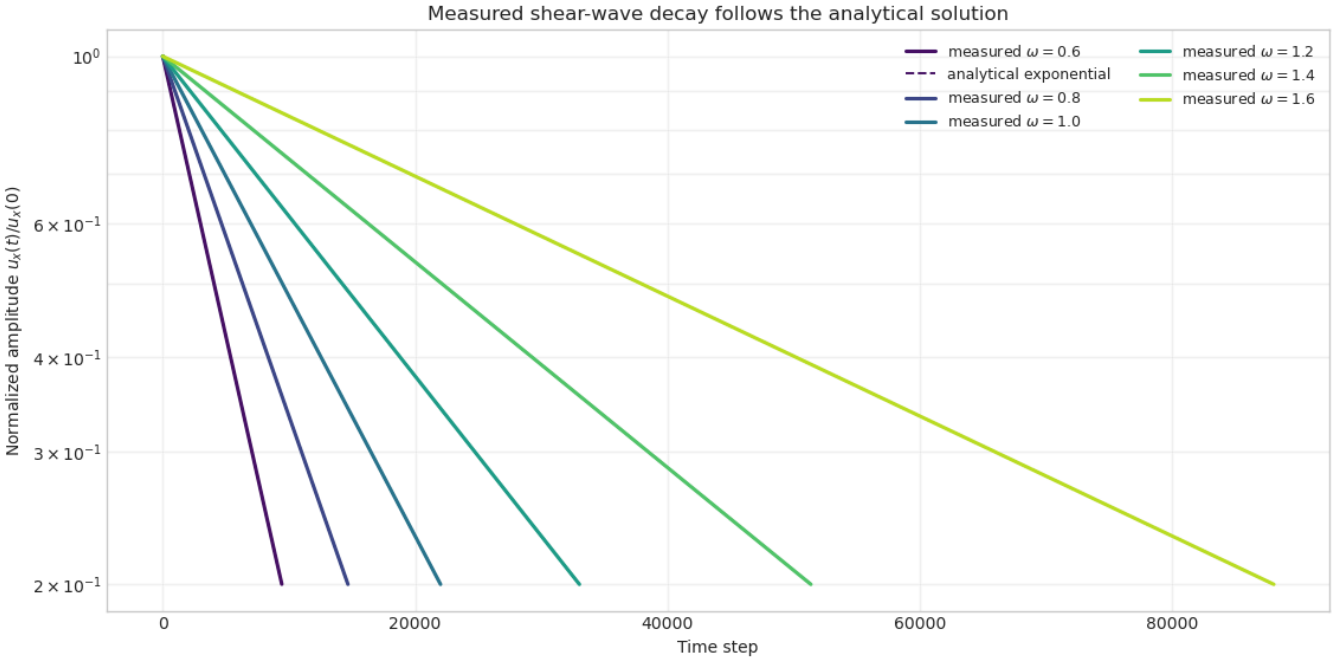
A periodic sinusoidal velocity mode should decay according to $u_x(t) = u_0 \exp(-\nu k^2 t)$. Fitting the simulated decay therefore measures the viscosity independently of the input relation $\nu = \frac{1}{3}(1/\omega - 1/2)$.



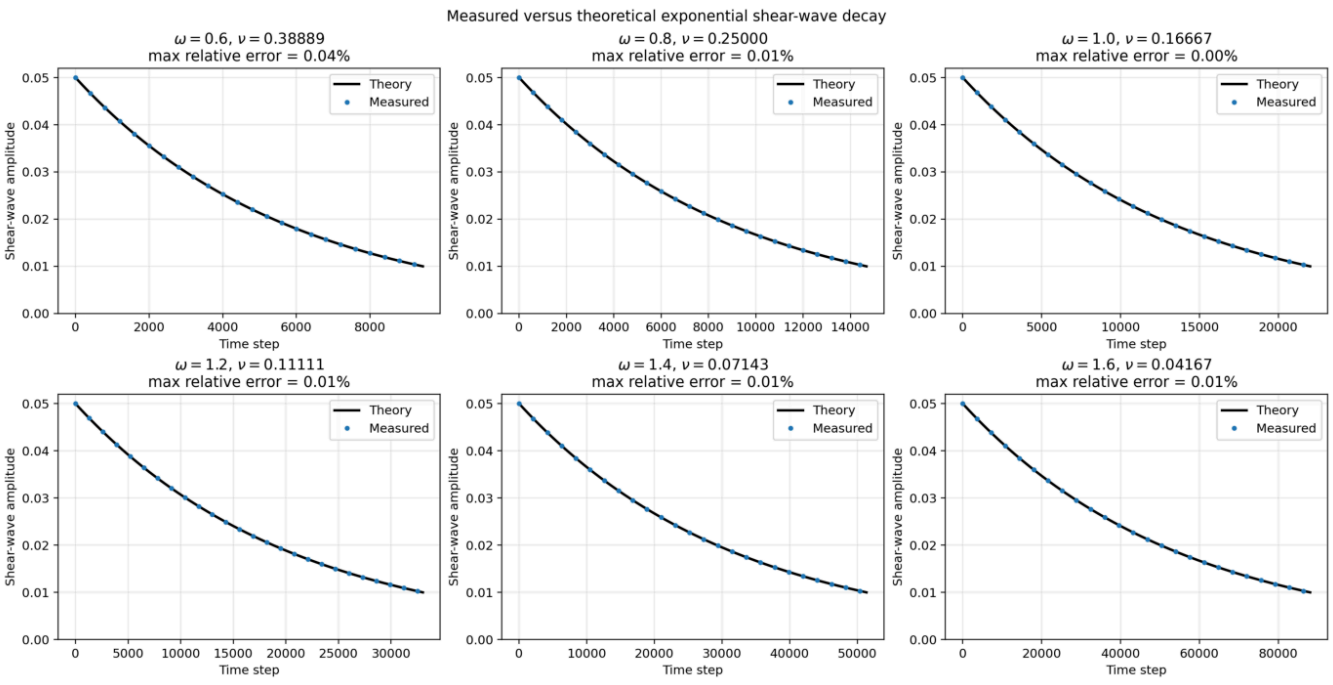

```
Worst relative viscosity error: 0.02634%
```

Shear-wave decay histories

The decay curves are the direct validation result behind the fitted viscosities. Measured amplitudes follow the analytical exponential across the full relaxation-rate sweep. Higher viscosity produces faster damping; the small separation visible at $\omega = 0.6$ is still only a 0.0264% viscosity error.



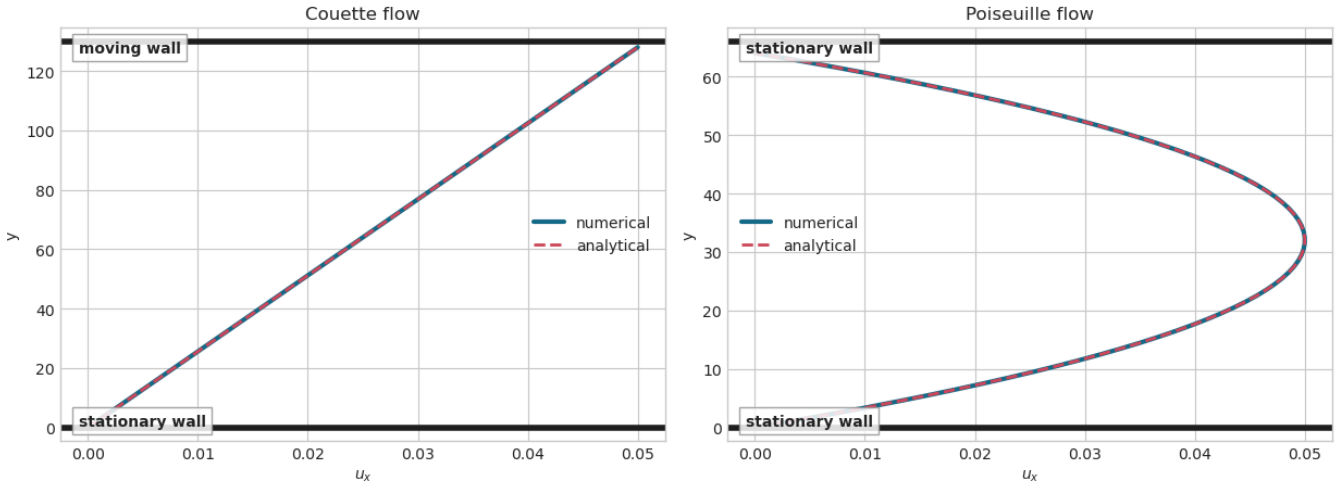
The panel view below presents each relaxation rate separately, as in the original presentation. This makes the agreement between measured samples and theory visible without normalization and reports the maximum pointwise relative error for every run.



2.2 Channel-flow profiles

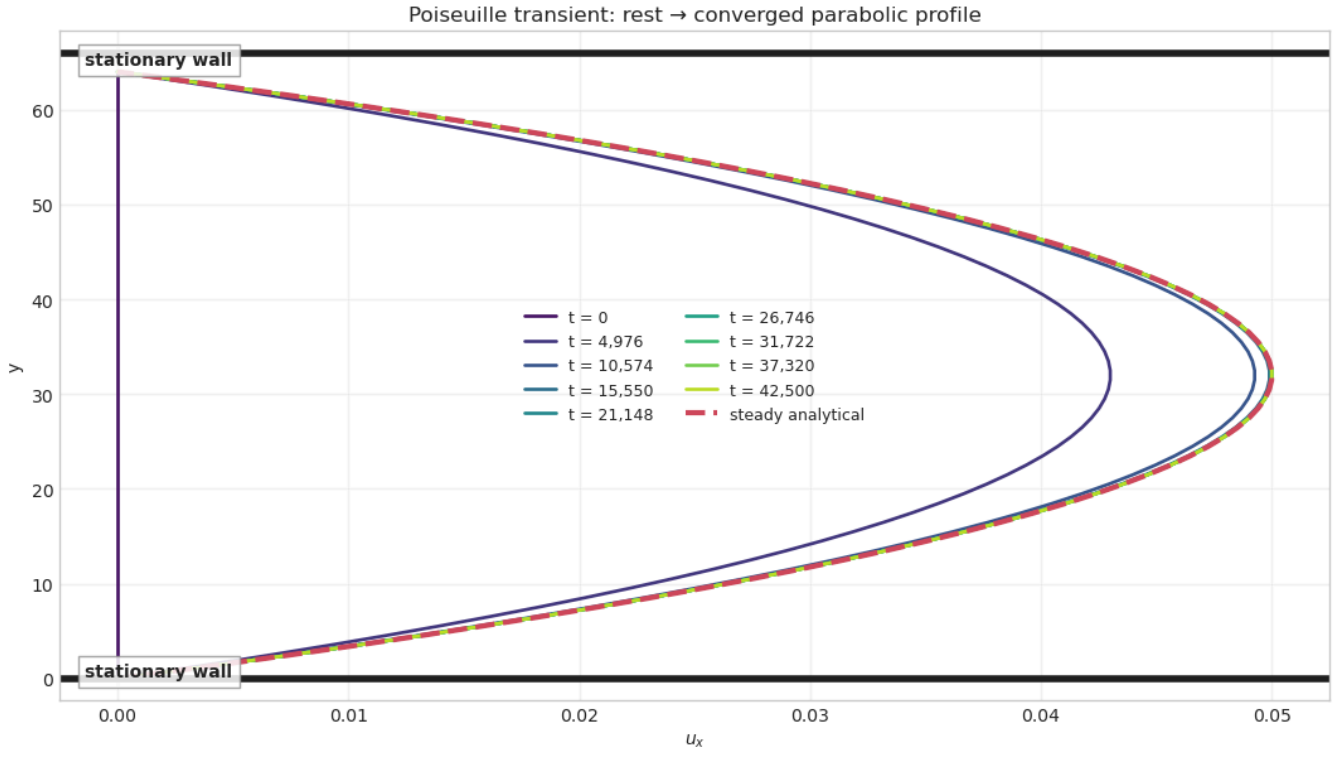
Couette flow tests the moving-wall boundary and should produce a linear velocity profile. Poiseuille flow tests stationary bounce-back plus Guo body forcing and should produce a parabola. Agreement in both cases checks not only collision and streaming, but also the

physical half-way wall location. The fluid nodes lie half a lattice spacing inside the channel; the heavy horizontal lines mark the actual walls at $y = 0$ and $y = L$.



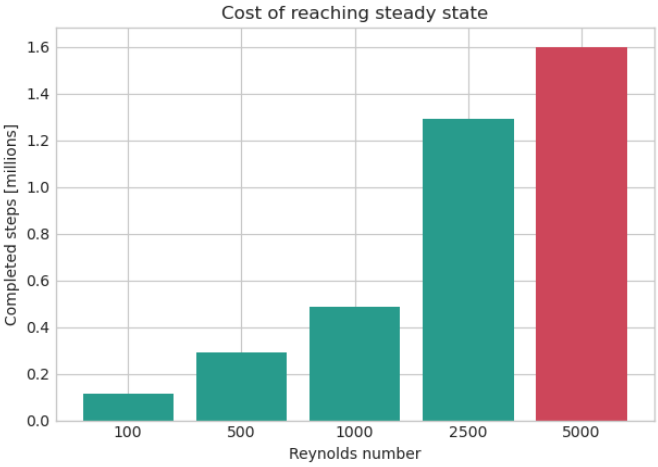
Poiseuille flow approaching the theoretical solution

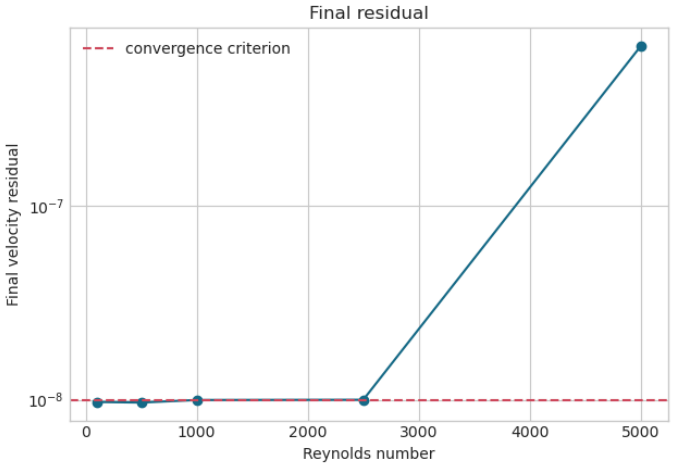
The body force starts accelerating the fluid from rest. Viscous momentum diffusion then carries the wall condition into the channel until the numerical profile becomes the steady analytical parabola. The curves show representative saved states from $t = 0$ through convergence at step 42,500; both stationary walls are drawn explicitly.



2.3 Lid-driven cavity convergence

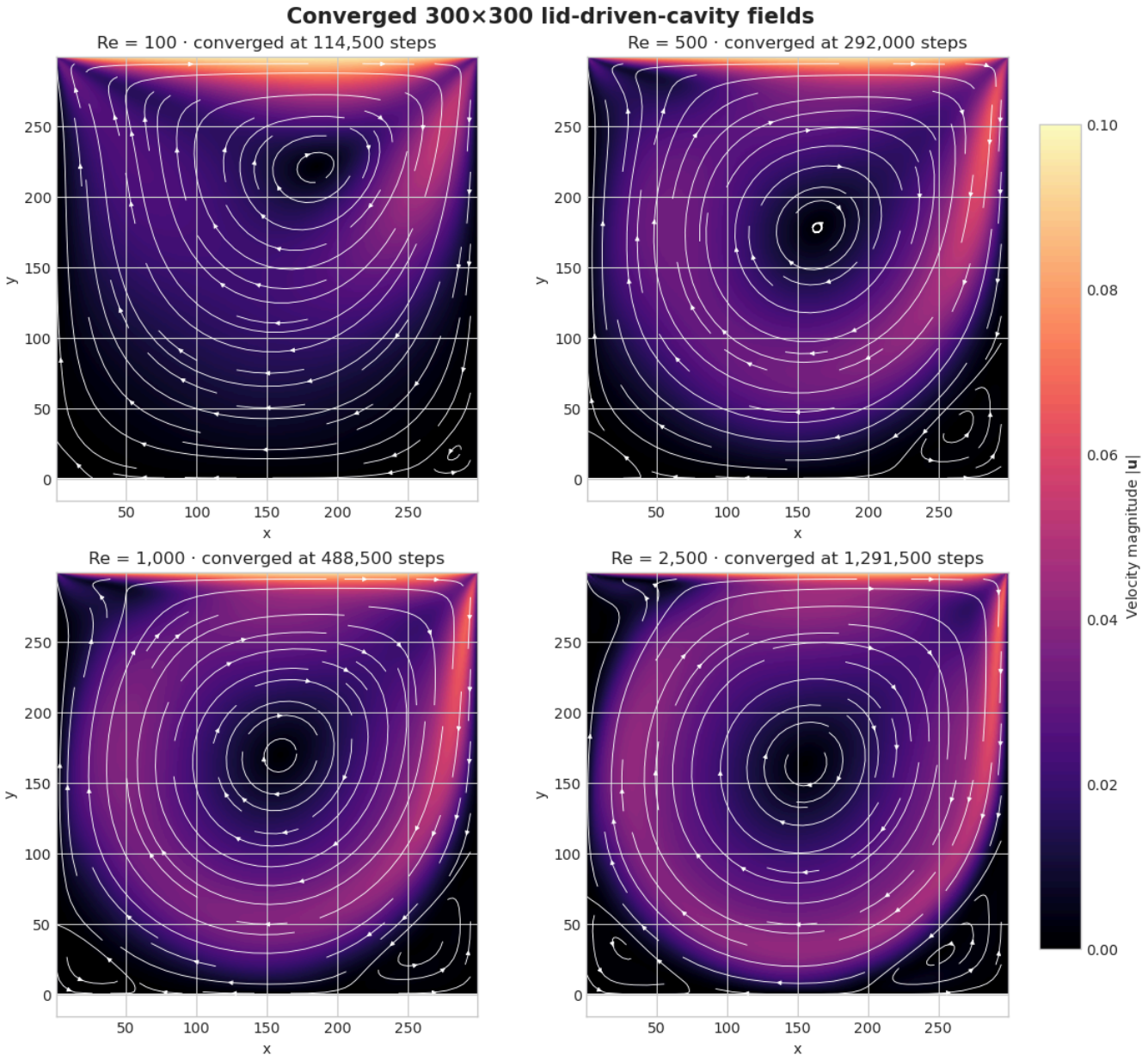
Increasing Reynolds number reduces viscosity and greatly increases the time required to reach a steady residual. Re=5000 did not meet the convergence criterion within the step limit, so its flow field is useful as a finite-time result but not as a converged steady benchmark.





Lid-cavity velocity fields

These converged fields are central physical results of the solver. The colour map shows velocity magnitude and the streamlines show the recirculating structure. As Reynolds number increases, the primary vortex becomes sharper and secondary corner vortices become more pronounced. Only Re=100, 500, 1000, and 2500 are shown because they satisfy the steady convergence criterion; Re=5000 remains a bounded finite-time calculation.



3. What was tested

Case	Principal test	Result
Shear wave	Viscous decay and $\omega \leftrightarrow \nu$ relation	Worst viscosity error 0.0264%
Density wave	Propagation, damping, and mass conservation	Mean-density error below 10^{-10} at saved slices
Couette	Moving wall and half-way bounce-back	Numerical and linear profiles agree
Poiseuille	Body force and stationary walls	Relative L2 error 1.11×10^{-4}
Lid cavity	Four-wall boundary interaction and steady convergence	Re≤2500 converged; Re=5000 bounded

Together these cases exercise periodic and physical boundaries, moving and stationary walls, body forcing, distributed halo exchange, convergence checks, and global output assembly.

4. Numerical method in brief

The solver stores nine D2Q9 populations at each lattice cell. A time step consists of:

1. computing density and velocity;
2. local BGK collision, with Guo forcing for Poiseuille flow;
3. exchanging one-cell post-collision halos between coarray images;
4. pull streaming from local or halo source cells;
5. reconstructing missing wall populations with half-way bounce-back.

The global grid is decomposed into a near-square image grid. Pull streaming gives each destination cell a single local writer, while remote source values are read from halo cells. This keeps the time-step kernel uniform across the interior and image boundaries.

The kinematic viscosity is controlled by the relaxation rate:

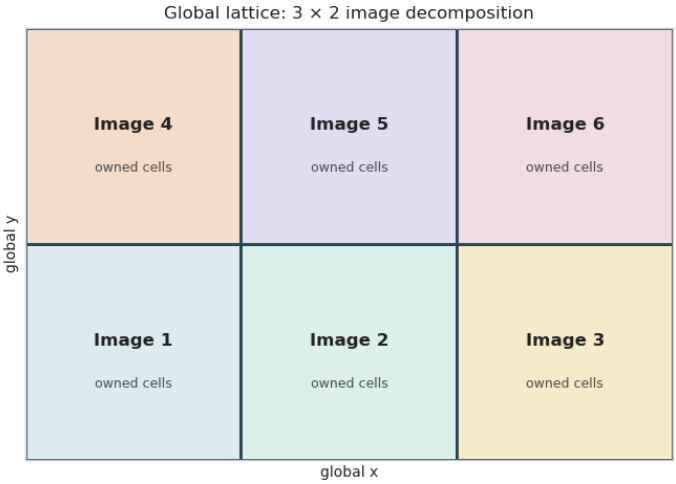
$$\nu = \frac{1}{3}\left(\frac{1}{\omega} - \frac{1}{2}\right), \qquad 0 < \omega < 2.$$

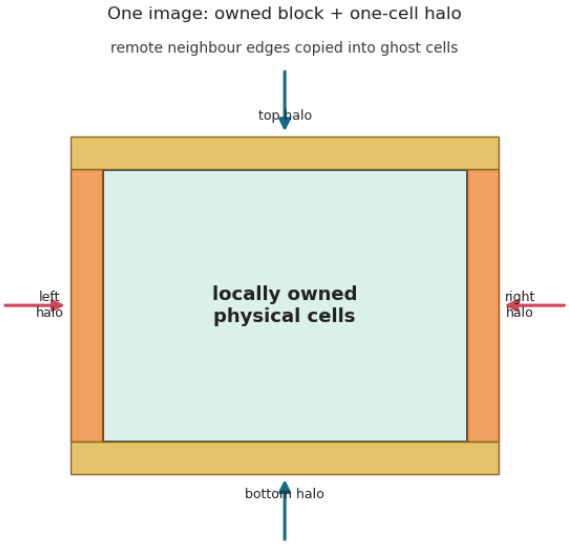
These details are sufficient to interpret the validation results; a fuller derivation and annotated Fortran excerpts remain in `lbm_code_presentation.ipynb`.

Domain decomposition and halo communication

The global lattice is divided over a near-square two-dimensional image grid. Each image owns a rectangular block of physical cells plus a one-cell ghost layer. After collision, an image reads its neighbours' edge populations into these halo cells. Pull streaming can then use the same local indexing for both local and remote sources.

The diagram distinguishes the global ownership map from one image's local allocation. Horizontal halos are filled first; top and bottom rows are then transferred including the newly filled corner ghosts.





The implementation first chooses a factorization that keeps local blocks close to square, computes each image's local extent and global offset, and then performs neighbour-scoped coarray reads:

```
call choose_image_grid(nimgs, px, py)
ix_img = mod(my_img - 1, px) + 1
iy_img = (my_img - 1) / px + 1
nx_loc = block_size(nx, px, ix_img)
ny_loc = block_size(ny, py, iy_img)
x_offset = block_offset(nx, px, ix_img)
y_offset = block_offset(ny, py, iy_img)

! Publish/read horizontal edges first.
sync images(neighbour_images(:neighbour_count))
if (left_img /= 0) then
  f(:,0,1:ny_loc) = f(:,remote_nx,1:ny_loc)[left_img]
end if
if (right_img /= 0) then
  f(:,nx_loc+1,1:ny_loc) = f(:,1,1:ny_loc)[right_img]
end if

! Rows include the new column ghosts, which also propagates corners.
sync images(neighbour_images(:neighbour_count))
if (up_img /= 0) then
  f(:,0:nx_loc+1,ny_loc+1) = f(:,0:nx_loc+1,1)[up_img]
end if
```

Square brackets identify a remote coarray image. The assignment itself still writes only into the current image's halo, preserving single ownership of streamed destinations.

5. Limitations

- Strong-scaling curves currently stop at 256 images and use only three repeats.

- The same 5,000-step duration produces very different wall times across grid sizes and image counts; the shortest measurements are vulnerable to startup and scheduler noise.
- No completed fixed-image placement study is present, so the cost of spreading the same number of images across more nodes is not yet quantified.
- The performance result measures the present implementation, including separate collision/streaming arrays and a full `f = f_new` copy. It is not a hardware roofline.
- The Re=5000 cavity case is not a converged steady solution.
- There is no external reference-profile comparison for the lid cavity (for example, centerline velocity data), so the lid result is presently a convergence and qualitative flow-structure check.

6. Framework development opportunities

The present BGK implementation is a useful baseline rather than a fixed endpoint. Several additions could broaden both its numerical robustness and the range of geometries it can represent:

- **Alternative collision operators.** Two-relaxation-time (TRT) or multiple-relaxation-time (MRT) collision would separate the relaxation of different kinetic modes instead of forcing all non-conserved modes to share the BGK rate. This can improve stability near $\omega \to 2$ and provide better control of wall accuracy. Entropic or regularized collision models are further options for demanding low-viscosity flows.
- **Internal stationary walls.** A solid/fluid mask and link-wise bounce-back could place obstacles and channels anywhere inside the lattice, rather than restricting walls to global domain edges. Interpolated bounce-back would improve the treatment of curved surfaces that do not lie halfway between nodes.
- **Internal moving boundaries.** The same link-based representation can be extended with a wall-velocity correction for translating or rotating objects. Momentum exchange would provide hydrodynamic forces and torques, enabling particle–fluid or moving-obstacle simulations.
- **More general geometry coupling.** For complex moving shapes, an immersed-boundary formulation could replace or complement bounce-back while retaining the existing coarray decomposition and halo infrastructure.

The collision step is local and the halo exchange already communicates population fields, so these developments can reuse the current streaming and parallel framework. Each addition would need dedicated analytical or reference-data validation.

7. Next cluster measurements

With limited time, these runs weren't completed but would be the most useful to further investigate how performance scales:

1. **Fixed-image placement study (highest value).** Use 384 total images on a 5000×5000 or 10000×10000 grid, spread across 16, 24, and 48 nodes, with at least three exclusive repeats. These placements use 24, 16, and 8 images per node and directly measure the trade-off between memory bandwidth/contention within a node and network communication between nodes.
2. **Longer, cleaner timing at the scaling frontier.** Extend the timed region until each sample lasts roughly 20–60 seconds. Repeat 5–7 times, especially at 128, 256, and 384 images. This will make efficiency claims much more defensible.
3. **Large-grid strong scaling.** Measure one larger fixed problem at 64, 128, 256, and 384 images. The existing 300×300 case is already saturated and should not be extended further.
4. **Cluster A/B test of halo synchronization.** Compare the old `sync all` version with the current neighbor-scoped `sync images` version using the same allocation and randomized run order. The local result suggests a benefit, but it does not establish the multi-node effect.

The repository already contains `submit_lid_scaling2.sh` and `lid_performance2.slurm` for the first experiment.